Online Image Description System

Assignment Report

Group project (Group 0XX) | November 2026

Group members

Name	Student ID	Role
[Member 1 name]	[Student ID]	Server implementation, testing
[Member 2 name]	[Student ID]	Client implementation, testing
[Member 3 name]	[Student ID]	Report, integration testing

1. Introduction

This report documents our implementation of the online image description system. The system follows a client-server design over one persistent TCP connection. The server listens on port 18333 by default, stores uploaded JPEG/PNG images in memory (at most 20 images and 50 MiB), and generates a two-sentence caption for a stored image with the locally hosted SmolVLM-256M-Instruct vision-language model. Every message on the wire is prefixed with a ten-digit ASCII length header, so every payload can be reconstructed exactly and nothing depends on newline framing.

Task 1 is the server: the five socket primitives (create_listener, accept_client, receive_message, send_message, close_client) and the already provided storage, AI and command handling around them. Task 2 is the client: a console program with six commands (UPLOAD, LIST, DESCRIBE, DOWNLOAD, REMOVE, QUIT) that keeps a single connection alive and displays every reply the server sends.

2. Runtime environment

The system runs on CPU only. A setup script (setup_env.bat) prepares an isolated environment with the pinned dependency set; no system-wide installations are required.

Component	Detail
Operating system	Windows 11 (25H2, build 26200), 64-bit
Shell	Windows PowerShell 5.1
Python	3.11.15 (uv-managed CPython, project-local .venv)
PyTorch	2.2.2+cpu (CPU wheel backend)
Transformers	4.49.0
Pillow	10.4.0

Component	Detail
Model	SmolVLM-256M-Instruct, stored locally in models\smolvlm-256m
Environment setup	setup_env.bat (uv 0.10.9, requirements.lock.txt)

No build step is required: both programs are Python sources executed with the project's .venv interpreter. The commands used to run the finished system are:

```
cd image-description-assignment
.\setup_env.bat
.\.venv\Scripts\python.exe ImageDescriptionServer_<student_IDs>.py
# second terminal (same folder):
.\.venv\Scripts\python.exe ImageDescriptionClient_<student_IDs>.py 127.0.0.1 18333
```

3. Task 1 - Server

3.1 The five socket functions

Function	Behaviour implemented
create_listener (host, port)	Creates an IPv4 (AF_INET) TCP (SOCK_STREAM) socket, enables SO_REUSEADDR, binds to (host, port), listens, and closes the socket again if any step raises OSError before re-raising.
accept_client (listener)	Blocks until a client connects and returns the pair (conn, address) produced by listener.accept().
receive_message (conn)	Reads exactly 10 header bytes; returns None when the peer closes the connection between messages. A header that is not 10 ASCII digits raises ValueError('Invalid length header'); a length outside 1..5,242,880 raises ValueError('Message size must be 1 byte to 5 MiB'). The payload is then read with a loop until the declared byte count is reached; if the connection closes first, ConnectionError is raised.
send_message (conn, payload)	Accepts bytes, bytearray or memoryview only (TypeError otherwise) with a length of 1..5,242,880 bytes (ValueError otherwise). Sends the zero-padded 10-digit ASCII length header followed by the payload with sendall.
close_client (conn)	Closes the connection socket (no exception on a second close).

3.2 Protocol and storage rules

Messages are framed as a 10-character decimal length header (zero padded) followed by exactly that many bytes. A command message is at most 1024 bytes. File names must be at most 255 characters and must not contain a path separator, a control character, or be '.' or '..'. Accepted images are JPEG or PNG, at most 5 MiB, and at most 20 megapixels; the server stores at most 20 images and 50 MiB in total. Image identifiers are decimal strings (0000, 0001, ...) that are never reused within one run of the server.

Command	Server reply
UPLOAD <filename> + second message with image bytes	OK <id>

Command	Server reply
LIST	OK, then one line per image: IMAGE ID: <id> | FILE: <name> | TIME: HH:MM:SS | DESCRIPTION: yes|no; or the single line No images
DESCRIBE <id>	OK and a new line with the caption (cached after first use)
DOWNLOAD <id>	OK <filename>, then a second message with the image bytes
REMOVE <id>	OK
QUIT	OK, then the connection is closed
invalid command, id, image, file name or full storage	ERROR - Command not understood | ERROR - Invalid ID | ERROR - Invalid image | ERROR - Invalid filename | ERROR - Image storage is full

3.3 Description backend

Uploaded images are decoded with Pillow, validated as JPEG/PNG below the megapixel limit, and normalised onto a white 512x512 canvas. DESCRIBE passes the image to SmolVLM-256M-Instruct with a fixed prompt asking for the main objects and the setting in two short sentences. The first description of an image is cached, so repeated DESCRIBE calls return the same text without running the model again; the LIST command reports DESCRIPTION: yes once a caption exists. The server can be started with --backend mock for protocol testing without loading the model, and with --check-ai <image> to verify the model installation on its own.

4. Task 2 - Client

The client is a single-file Python program that uses only the standard library (socket, sys, os, pathlib). It opens one IPv4 TCP connection to the host and port given on the command line and keeps it open for the whole session. The input prompt is written as client: on its own line, and every line received from the server is printed with the server: prefix. Message framing mirrors the server: the client writes the zero-padded 10-digit header, then the payload, and reads replies with the same exact-length loop.

Command	Client behaviour
UPLOAD	Prompts for an image path. The file must exist, be a regular file of 1 byte to 5 MiB, and be readable; otherwise the client answers locally with ERROR - File not found or ERROR - Image must be an existing file of 1 byte to 5 MiB without contacting the server. Only the base name is sent, followed by a second message containing the image bytes.
LIST	Sends LIST and prints every reply line prefixed with server:
DESCRIBE	Prompts for an image id, sends DESCRIBE <id>, prints the caption
DOWNLOAD	Prompts for an image id and an output path. The client refuses an existing file or a missing parent directory with ERROR - Choose a new file in an existing directory. The file is created exclusively and the client prints Saved to <path> only after the complete image was written.

Command	Client behaviour
REMOVE	Prompts for an image id, sends REMOVE <id>, prints OK
QUIT	Sends QUIT, prints the reply and closes the connection with exit code 0
anything else	Forwarded to the server unchanged so the user sees its ERROR reply; the connection stays open.

Error handling keeps the session usable: a server ERROR reply is shown and the same connection continues. Errors the client detects by itself (missing file, wrong size, existing download target) are reported without sending anything. End of input triggers a best-effort QUIT and exit code 0; an unreachable server exits with code 2; a connection lost during the session exits with code 1.

5. Verification

5.1 Automated server tests (check_sockets.py)

check_sockets.py imports the finished server file, replaces only the AI backend with a stub, and drives the five socket functions and the provided command loop over real TCP sockets on 127.0.0.1. The run reported 40 of 40 checks passed with exit code 0. Covered behaviour: listener creation, accept, framing of payloads from 1 byte to 5,242,880 bytes, a header delivered byte by byte, two messages arriving in one read, a malformed header, a zero and an oversized length, clean EOF, a truncated payload, rejected send arguments, second close, and the full command flow (UPLOAD, LIST, DESCRIBE with caching, DOWNLOAD, REMOVE, QUIT, unknown and overlong commands).

5.2 Client conformance test (client_harness.py)

client_harness.py starts the server with the mock backend on an ephemeral port and scripts a complete session through the client's standard input, then compares the whole transcript, the downloaded file and the exit codes with the expected behaviour. The run finished with PASS and exit code 0. The verified transcript is:

```
client: UPLOAD
client: valid.png
server: OK 0000
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: valid.png | TIME: HH:MM:SS | DESCRIPTION: no
client: DESCRIBE
client: 0000
server: OK
server: Mock description: the image was received and decoded successfully.
client: DESCRIBE
client: 9999
server: ERROR - Invalid ID
client: DOWNLOAD
client: 0000
client: out.png
server: OK valid.png
client: Saved to out.png
client: REMOVE
client: 0000
server: OK
client: QUIT
server: OK
```

The same harness also verified the local pre-checks (missing and invalid image paths), the refusal to overwrite an existing download target, the byte-for-byte equality of the downloaded file, and exit code 2 when the server is not running.

5.3 Real-AI end-to-end run (e2e_real_ai.py)

The end-to-end script starts the unmodified server with the real model, uploads the three provided test images through the finished client, describes each of them, downloads one image and removes another. The run passed with exit code 0; the transcript was:

```
client: UPLOAD
client: cat.jpg
server: OK 0000
client: UPLOAD
client: flower.png
server: OK 0001
client: UPLOAD
client: fruit_bowl.jpg
server: OK 0002
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: cat.jpg | TIME: 10:25:20 | DESCRIPTION: no
server: IMAGE ID: 0001 | FILE: flower.png | TIME: 10:25:20 | DESCRIPTION: no
server: IMAGE ID: 0002 | FILE: fruit_bowl.jpg | TIME: 10:25:20 | DESCRIPTION: no
client: DESCRIBE
client: 0000
server: OK
server: In this image we can see a cat lying on the floor.
client: DESCRIBE
client: 0001
server: OK
server: In this image we can see two pink roses and two buds.
client: DESCRIBE
client: 0002
server: OK
server: In this image there are apples, oranges, pears, and a white bowl.
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: cat.jpg | TIME: 10:25:20 | DESCRIPTION: yes
server: IMAGE ID: 0001 | FILE: flower.png | TIME: 10:25:20 | DESCRIPTION: yes
server: IMAGE ID: 0002 | FILE: fruit_bowl.jpg | TIME: 10:25:20 | DESCRIPTION: yes
client: DOWNLOAD
client: 0002
client: fruit_bowl_copy.jpg
server: OK fruit_bowl.jpg
client: Saved to fruit_bowl_copy.jpg
client: REMOVE
client: 0001
server: OK
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: cat.jpg | TIME: 10:25:20 | DESCRIPTION: yes
server: IMAGE ID: 0002 | FILE: fruit_bowl.jpg | TIME: 10:25:20 | DESCRIPTION: yes
client: QUIT
server: OK
```

5.4 Screenshots from our own runs

The screenshots below were captured on the machine described in Section 2. They show the environment being checked, the server printing its READY line, and a complete interactive session.

```
PS C:\WINDOWS\System32> cd "C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment"
PS C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment> .\.venv\Scripts\python.exe ImageD
escriptionServer.py --check-ai .\test_images\cat.jpg
In this image we can see a cat lying on the floor.
PS C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment>
```

Figure 1. The environment check: --check-ai prints a caption for test_images\cat.jpg.

```
In this image we can see a cat lying on the floor.
PS C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment> .\.venv\Scripts\python.exe ImageD
escriptionServer.py
READY host=0.0.0.0 port=18333
```

Figure 2. The server window after startup: READY host=0.0.0.0 port=18333.

```
PS C:\Users\qwe11> cd "C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment"
PS C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment> .\.venv\Scripts\python.exe "C:\Users\qwe11\image-description-
final\ImageDescriptionClient_STUDENT_IDS.py" 127.0.0.1 18333
client: UPLOAD
client: test_images\fruit_bowl.jpg
server: OK 0001
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: fruit_bowl.jpg | TIME: 14:05:42 | DESCRIPTION: yes
server: IMAGE ID: 0001 | FILE: fruit_bowl.jpg | TIME: 14:12:28 | DESCRIPTION: no
client: DESCRIBE
client: 0000
server: OK
server: In this image there are apples, oranges, pears, and a white bowl.
client: DESCRIBE
client: 0001
server: OK
server: In this image there are apples, oranges, pears, and a white bowl.
client: DOWNLOAD
client: 0000
client: fruit_bowl_copy.jpg
server: OK fruit_bowl.jpg
client: Saved to fruit_bowl_copy.jpg
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: fruit_bowl.jpg | TIME: 14:05:42 | DESCRIPTION: yes
server: IMAGE ID: 0001 | FILE: fruit_bowl.jpg | TIME: 14:12:28 | DESCRIPTION: yes
client: REMOVE
client: 0000
server: OK
client: LIST
server: OK
server: IMAGE ID: 0001 | FILE: fruit_bowl.jpg | TIME: 14:12:28 | DESCRIPTION: yes
client: QUIT
server: OK
```

Figure 3. A complete interactive client session: UPLOAD, LIST, DESCRIBE, DOWNLOAD, REMOVE, QUIT.

```
PASS  QUIT replies OK
PASS  the handler closes the connection after QUIT
PASS  the handler thread finished
PASS  a malformed header gets an ERROR reply
PASS  the connection ends after a malformed header
PASS  an over-long command gets an ERROR reply

40/40 checks passed
PS C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment> .\.venv\Scripts\python.exe "C:\Users\qwe11\image-description-
final\tests\client_harness.py" --server ".\ImageDescriptionServer.py"
server ready on 127.0.0.1:64421
PASS: client transcript, downloads and error handling verified
PS C:\Users\qwe11\Downloads\image-description-assignment\image-description-assignment>
```

Figure 4. check_sockets.py (40/40) and client_harness.py (PASS) results.

6. Discussion

The length-prefixed framing removed the usual newline and partial-read problems and let the same helpers serve both directions. Storing only the required metadata meant that the listing was cheap, while caching descriptions kept repeated DESCRIBE calls fast on CPU. The main limitation is caption quality: the 256M model describes the prominent objects correctly but occasionally misses details such as small text or exact counts. Upload, download and removal were validated with the provided demo client and against our own client, so both ends accept the same byte stream.

7. Conclusion

Both tasks are complete: the five socket functions give the server reliable, exact message exchange, and the client implements the six commands with the required prompts, prefixes, error messages and exit codes

over a single persistent connection. Automated checks confirm the protocol boundaries, and a full run on the real model shows the system describing uploaded images end to end.

Notes (remove before submission)

Replace the member table and the [Student ID] placeholders, then rename the three files to the submission pattern ImageDescriptionServer_<student_IDs>.py, ImageDescriptionClient_<student_IDs>.py and Report_<student_IDs>.pdf. Submit only the sources and this PDF; do not upload the model directory, the .venv, the .tools directory, setup.log or any executables.